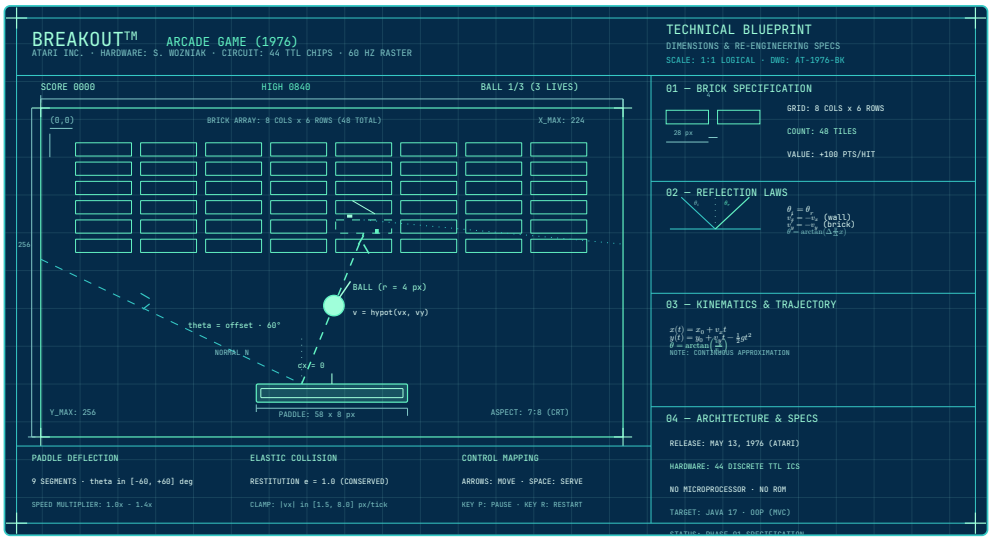
دانشگاه تهران
دانشکده علوم کامپیوتر

برنامه‌سازی پیشرفته

فاز پروژه

۱ هسته بازی Breakout

اهداف، نیازمندی‌های فنی، طراحی شی‌گرا و معیارهای ارزیابی



«در سال ۱۹۷۶، نولان باشنل در شرکت آتاری از استیو جابز خواست بازی Breakout را با کمترین تعداد تراشه سخت‌افزاری بسازد. جابز پروژه را به استیو وازنیاک سپرد؛ وازنیاک طی ۴ شبانه‌روز کار بی‌وقفه، مدار بازی را تنها با ۴۴ تراشه پیاده کرد. ابتکارات وازنیاک در بهینه‌سازی مدارهای گرافیکی این بازی، مستقیماً الهام‌بخش او در طراحی معماری نمایش تصویر و رنگ در رایانه انقلابی Apple II شد.»

پرامپت پیشنهادی برای بازتولید تصویر شماتیک مهندسی

Technical blueprint drawing of classic 1976 Breakout arcade game, showing green phosphor wireframe paddle, mathematical ball deflection angles, and brick array grid on deep navy blueprint paper, clean line art, precise dimension labels, engineering schematic style.

فهرست مطالب

قوانین و شرایط تحویل ۲
استقلال در پیاده‌سازی و مالکیت کد ۲
ثبت پیوسته در گیت‌هاب (GitHub) ۲
جلسه دفاع و گفت‌وگوی فنی ۲
بستر تحویل و موعد نهایی ۲
استفاده از ابزارهای هوش مصنوعی ۲
معرفی بازی و اهداف ۳
اهداف اصلی فاز اول ۳
اقدامات پیش از شروع ۴
۱. راه‌اندازی گیت‌هاب و ساختار پروژه ۴
۲. تجربه بازی و درک رفتار فیزیکی ۴
۳. طراحی مدل شی‌گرا (نمودار UML) ۴
ساختار و معماری سیستم ۵
جریان داده در ۳ گام ۵
قرارداد ذخیره رکورد (`HighScoreStorage`) ۵
موجودیت‌ها و هندسه بازی ۶
اصول طراحی شی‌گرا و کپسوله‌سازی ۶
سازوکار گیم‌پلی و وضعیت‌ها ۷
گرافیک و واسط کاربری ۸
۱. دستگاه مختصات و هندسه بوم ۸
۲. پایداری تصویر و بافر دوگانه (Double Buffering) ۸
۳. اجزای دیداری و نوار وضعیت ۸
فیزیک و برخوردها ۹
۱. بازتاب زاویه‌دار از سکو ۹
۲. تشخیص برخورد با آجرها (AABB) ۹
گام‌های پیاده‌سازی ۱۰
راهبرد توسعه و کامیت‌ها ۱۰
معیارهای ارزیابی و دفاع ۱۱
جلسه دفاع و گفت‌وگوی فنی ۱۱
بخش‌های امتیازی و اختیاری ۱۱

قوانین و شرایط تحویل

پروژه درس برنامه‌سازی پیشرفته یک ماجراجویی فردی و جذاب است تا آموخته‌های شی‌گرایی را در قالب یک نرم‌افزار واقعی تجربه کنید. برای اینکه این مسیر یادگیری لذت‌بخش و عادلانه باشد، چند اصل ساده و حرفه‌ای را با هم مرور می‌کنیم:

استقلال در پیاده‌سازی و مالکیت کد

این پروژه کاملاً **فردی** است و هدفش ساختن اعتمادبه‌نفس فنی شماست. تمام کدی که می‌نویسید باید حاصل درک مستقیم خودتان باشد. مطالعه منابع، راهنماها و مقالات برای یادگیری بسیار خوب است؛ اما کپی‌کردن کد دیگران یا ارسال کدهای ماشینی بدون تسلط کامل، با اهداف آموزشی درس در تضاد است. در جلسه دفاع باید بتوانید فلسفه پشت هر خط کد را با تسلط کامل توضیح دهید.

ثبت پیوسته در گیت‌هاب (GitHub)

فرآیند تفکر مهندسی شما از طریق کامیت‌های مرحله‌به‌مرحله (Git Commits) در مخزن گیت‌هاب سنجیده می‌شود. کدنویسی یک جریان آرام و پیوسته است؛ بنابراین قابلیت‌ها را گام‌به‌گام اضافه کرده و با پیام‌های تمیز ثبت کنید. از ارسال ناگهانی کل پروژه در قالب یک کامیت در ساعات پایانی بپرهیزید.

جلسه دفاع و گفت‌وگوی فنی

پس از تحویل پروژه، در یک مصاحبه حضوری دوستانه با دستیاران آموزشی، کد شما بررسی می‌شود. در این نشست، تسلط شما بر معماری، کلاس‌ها و نحوه پیاده‌سازی ارزیابی شده و نمره نهایی پروژه تثبیت می‌گردد.

بستر تحویل و موعد نهایی

برای تحویل نهایی، نشانی مخزن گیت‌هاب پروژه را پیش از پایان موعد مقرر در سامانه کوئرا (Quera) بارگذاری نمایید. دقت کنید که مخزن عمومی باشد یا دسترسی آن برای تیم تدریس باز شده باشد.

استفاده از ابزارهای هوش مصنوعی

هوش مصنوعی مثل یک منتور صبور است که می‌توانید برای فهم عمیق‌تر مفاهیم، اشکال‌زدایی و مشورت در طراحی از آن کمک بگیرید. با این حال، تصمیم‌گیرنده نهایی و مهندس اصلی سیستم شما هستید و تمام منطق بازی باید تحت تسلط کامل شما باشد.

همراهی دستیاران آموزشی

ما همیشه کنارتان هستیم تا مسیر یادگیری برایتان دلنشین‌تر شود. اگر در هر کجای معماری یا پیاده‌سازی با چالش یا ابهامی روبه‌رو شدید، در تالار گفتگو و جلسات رفع اشکال منتظر پرسش‌هایتان هستیم.

معرفی بازی و اهداف

در سال ۱۹۷۶، شرکت آتاری با ساخت بازی تعاملی بریک‌اوت (Breakout) نقطه عطفی در تاریخ بازی‌های ویدئویی ساخت. داستان جذاب پشت این بازی، هنرنمایی استیو وازنیاک (Steve Wozniak) بود که مدار سخت‌افزاری آن را تنها با ۴۴ تراشه منطقی بست؛ نبوغی که بعدها مستقیماً الهام‌بخش او در طراحی معماری تصویر و رنگ رایانه جریان‌ساز اپل دو (Apple II) شد.

امروز بریک‌اوت از دوست‌داشتنی‌ترین پروژه‌ها برای لمس واقعی مهندسی نرم‌افزار شی‌گرا و سیستم‌های تعاملی است: یک سکوی چابک در پایین صفحه، توپی پرانرژی که میان دیواره‌ها کمانه می‌کند، و دیواری از آجرهای رنگارنگ که منتظر نشانه‌گیری دقیق شما هستند!

اهداف اصلی فاز اول

در این فاز می‌خواهیم با هم یک هسته تمیز، پایدار و استاندارد به زبان جاوا (Java) بسازیم. مهم‌ترین مهارت‌هایی که در این مسیر تمرین می‌کنید:

- **طراحی شی‌گرا و تفکیک دغدغه‌ها (SoC):** جدا نگه‌داشتن دنیای ریاضی و قوانین برخورد بازی از دنیای نقاشی و رندرینگ گرافیکی.
- **اصل تک‌مسئولیتی (SRP):** سپردن یک وظیفه مشخص و تمیز به هر کلاس، و پرهیز از ساختن کلاس‌های هیولایی همه‌کاره (God Class).
- **کپسوله‌سازی شایسته (Encapsulation):** مخفی‌کردن متغیرهای وضعیت درونی و کنترل تغییرات صرفاً از طریق توابع با نام‌گذاری شفاف.
- **مدیریت روان رخدادها (Event-Driven):** گوش‌دادن بی‌درنگ به کلیدهای صفحه‌کلید و نوسازی پیوسته فریم‌ها در حلقه اصلی بازی.
- **معماری منعطف برای آینده:** طراحی کلاس‌ها به شکلی که در فاز بعد بتوان ذخیره رکورد در فایل و انواع پاورآپ‌ها را مثل آب خوردن به آن اضافه کرد.

محدوده مورد انتظار در فاز اول

خیالتان راحت باشد! هدف این فاز سلامت ساختار کد، فیزیک درست برخوردها و روانی گیم‌پلی است. نیازی به منوهای پیچیده، صداگذاری، یا گرافیک‌های سه‌بعدی نیست؛ یک واسط بصری ساده، تمیز و روان بالاترین نمره را برای شما به همراه خواهد داشت.

اقدامات پیش از شروع

قبل از اینکه سراغ نوشتن اولین خط کد در جاوا بروید، برداشتن ۳ گام مقدماتی زیر مسیر را برایتان کاملاً هموار و لذت‌بخش می‌کند:

۱. راه‌اندازی گیت‌هاب و ساختار پروژه

نخستین قدم، ایجاد یک مخزن در گیت‌هاب (GitHub) است. یک پروژه حرفه‌ای با ساختار پوشه‌بندی استاندارد و ثبت منظم تاریخچه تغییرات شناخته می‌شود:

- **ساختار پوشه‌بندی استاندارد:** کلاس‌های پروژه را در بسته‌های مجزا مرتب کنید:
  - پوشه `src/model/`: کلاس‌های وضعیت و منطق بازی (`Ball`، `Paddle`، `Brick` و `GameState`).
  - پوشه `src/view/`: کدهای رندر و واسط گرافیکی (`BreakoutCanvas` و `HUD`).
  - پوشه `src/controller/`: موتور هدایت و شنونده کلیدها (`GameEngine` و `Main`).
  - پوشه `src/util/`: مقادیر ثابت و ابزارها (`GameConstants`).
- **تنظیم گیت ایگنور برای تمیزی مخزن:**
  فایل‌های موقت، خروجی‌های کامپایل `.class` و تنظیمات ویرایشگر را درون فایل `.gitignore` بگذارید تا مخزن گیت‌هاب شما همیشه سبک و پاکیزه بماند.
- **نگارش فایل راهنما (README.md):**
  در این فایل نام و شماره دانشجویی خودتان، نسخه جاوا و دستور اجرای بازی را ثبت کنید.
- **کامیت‌های استاندارد (Conventional Commits):**
  تغییرات را در گام‌های کوچک و شفاف کامیت کنید؛ مانند `feat: add paddle` یا `fix: ball bounce`.

۲. تجربه بازی و درک رفتار فیزیکی

برای اینکه درک درستی از حس بازی و مکانیک‌های آن پیدا کنید، حتماً قبل از شروع کدنویسی چند دقیقه یک نسخه آنلاین از بریک‌اوت را تجربه کنید:

زاویه کمانه توپ

توپ در مرکز سکو بازتابی ملایم دارد و در کناره‌ها با زاویه تند پرتاب می‌شود.

روانی حرکت (۶۰ هرتز)

پاسخ آنی سکو به کلیدها و حرکت روان و نرم توپ بدون لگ یا پرش تصویری.

رفتار برخورد آجرها

محو شدن سریع آجر پس از اصابت، بازتاب لحظه‌ای توپ و ثبت امتیاز ۱۰۰ عددی.

۳. طراحی مدل شی‌گرا (نمودار UML)

یک طراحی خوب روی کاغذ، جلوی ساعت‌ها سردرگمی در کدنویسی را می‌گیرد:

ترسیم نمودار کلاس (UML Diagram)

پیشنهاد می‌کنیم با ابزارهایی مثل Draw.io، نام کلاس‌ها، فیلدهای خصوصی و متدهای عمومی را قبل از نوشتن کد مشخص کنید تا نقشه ذهنی روشنی داشته باشید.

وضعیت در ارزیابی فاز اول

تحویل نمودار در فاز اول **اختیاری** است؛ اما رسم آن به تسلط شخصی شما در جلسه مصاحبه دفاع کمک شایانی خواهد کرد.

ساختار و معماری سیستم

تصور کنید تمام بازی در یک کلاس غول‌آسای ۲۵۰۰ خطی نوشته شود: از سرعت توپ و برخورد با دیواره تا رنگ صورتی آجرها و شنونده کلید Space! شاید اولش ساده به نظر برسد، اما با اولین باگ نامرئی، کد تبدیل به کلافی سردرگم می‌شود. نجات‌بخش ما در این بازی، الگوی درخشان معماری مدل-نما-کنترل‌گر (MVC) است که وظایف را به ۳ بخش مستقل تقسیم می‌کند:

لایه مدل (Model) کلاس‌های منطق خالص	حقیقت جهان بازی؛ مختصات توپ، سکو، آجرها و جان‌ها. کدهای کاملاً مستقل از پنجره و گرافیک با آزمون‌پذیری آسان.
لایه نما (View) `BreakoutCanvas`	پرده نقاشی و تماشا؛ دریافت وضعیت از کنترل‌گر و ترسیم اشکال، آجرها و قلب‌های جان. بدون داشتن منطق بازی.
لایه کنترل‌گر (Controller) `GameEngine`	مغز هماهنگ‌کننده؛ اجرای حلقه بازی در ۶۰ هرتز، دریافت فشرده‌شدن کلیدها، حل برخوردها و فرمان بازترسیم به نما.

جریان داده در ۳ گام

۱. **ورودی کاربر:** فشرده شدن کلیدهای جهت‌نما یا کلیدهای فاصله، P و R توسط کنترل‌گر دریافت می‌شود.
۲. **محاسبه در مدل:** کنترل‌گر متدهای حرکتی سکو و توپ را صدا زده و وضعیت جدید دنیای بازی را ثبت می‌کند.
۳. **بازترسیم نما:** کنترل‌گر پس از قطعی شدن محاسبات فیزیک، وضعیت تازه را برای ترسیم به بوم نقاشی می‌فرستد.

قرارداد ذخیره رکورد (`HighScoreStorage`)

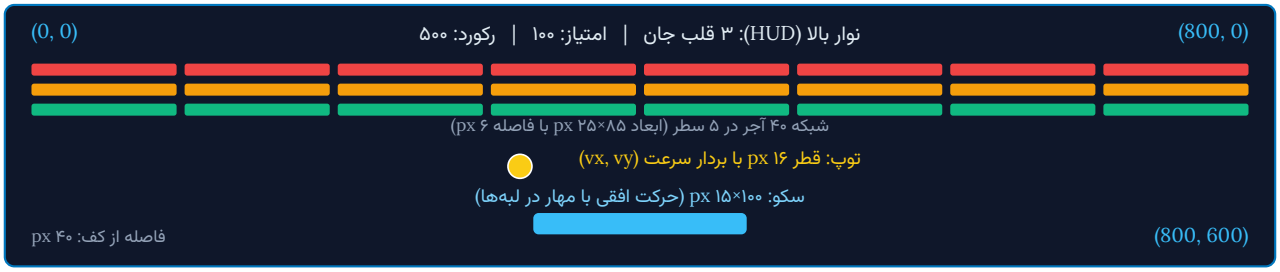
برای استقلال کامل بازی از نحوه ذخیره‌سازی، یک رابط ساده تعریف می‌کنیم تا در این فاز رکورد در رم بماند و در فاز آینده بدون دست زدن به موتور بازی، ذخیره در فایل متصل شود:

```
public interface HighScoreStorage {
    int loadHighScore();
    void saveHighScore(int score);
}
```

- **پیاده‌سازی موقت:** کلاس InMemoryScoreStorage برای نگه‌داری بالاترین رکورد در حافظه جاری بازی.
- **پیاده‌سازی آینده:** کلاس FileScoreStorage برای ذخیره دائمی رکورد در فایل متنی پس از آموزش I/O.

موجودیت‌ها و هندسه بازی

جهان بریک‌اوت از ۴ موجودیت اصلی تشکیل شده که هر کدام قوانین و هندسه خاص خود را دارند. مختصات صفحه از گوشه بالا-چپ $(0, 0)$ آغاز می‌شود و تمام ابعاد بر حسب پیکسل محاسبه می‌گردند:



سکو (`Paddle`)

مستطیلی به ابعاد ۱۰۰ در ۱۵ پیکسل در فاصله ۴۰ پیکسلی از پایین صفحه. جابه‌جایی منحصراً افقی با کلیدهای چپ و راست، همراه با تابع مهار موقعیت برای جلوگیری از خروج از کادر.

توپ (`Ball`)

دایره‌ای به قطر ۱۶ پیکسل (شعاع ۸) با بردار سرعت پیوسته. در شروع بازی و پس از سوختن، روی مرکز سکو می‌نشیند و تنها با زدن کلید فاصله پرتاب می‌شود.

آجرها (`Brick`)

شبکه‌ای در ۵ سطر و ۸ ستون (مجموعاً ۴۰ آجر) به ابعاد ۸۵ در ۲۵ پیکسل با فاصله ۶ پیکسل. با اصابت توپ، آجر محو شده و ۱۰۰ امتیاز به بازیکن داده می‌شود.

نشانگر وضعیت (HUD)

نوار بالایی صفحه که در جریان بازی ۳ قلب جان و امتیاز را نشان می‌دهد و پس از برد یا باخت، پیام پایان و رکورد ذخیره‌شده را به تصویر می‌کشد.

اصول طراحی شی‌گرا و کپسوله‌سازی

- **فیلدهای کاملاً خصوصی:** تعریف تمام متغیرهای مختصات، ابعاد و بردار سرعت با سطح دسترسی `private` تا هیچ کلاسی نتواند مستقیماً حالت درونی شیء دیگر را دستکاری کند.
- **ارائه رفتارهای معنادار:** پیاده‌سازی متدهایی با منطق روشن مانند حرکت به چپ، بازتاب عمودی و بررسی برخورد به جای قرار دادن توابع تغییردهنده خام.
- **ثابت‌های نام‌گذاری‌شده:** تجمیع ابعاد پنجره، سرعت پایه توپ، امتیاز هر آجر و فواصل در کلاس `GameConstants` برای پرهیز جدی از اعداد جادویی (Magic Numbers).

سازوکار گیم‌پلی و وضعیت‌ها

مدیریت جریان بازی با یک ماشین وضعیت متناهی و بسیار شفاف پیاده‌سازی می‌شود تا واکنش بازی به کلیدها، برخوردها و باخت جان‌ها کاملاً منظم و قابل پیش‌بینی باشد:



وضعیت	رفتار و انتقال‌های مجاز در جریان بازی
حالت آماده‌باش `READY`	وضعیت نخستین بازی یا پس از هربار سوختن یک جان؛ توپ در مرکز سکو مهار شده و همراه با آن جابه‌جا می‌شود. با فشردن کلید فاصله، بردار سرعت اولیه فعال شده و بازی وارد وضعیت اجرا می‌شود.
حالت اجرای فعال `RUNNING`	حلقه اصلی شبیه‌سازی بلادرنگ؛ حرکت پیوسته توپ، واکنش سکو به کلیدهای چپ و راست، بازتاب از دیواره‌ها و انهدام آجرها در این حالت انجام می‌پذیرد.
توقف موقت `PAUSED`	با فشردن کلید پی شبیه‌سازی متوقف و پیام توقف موقت روی صفحه نمایان می‌شود. فشردن دوباره همین کلید، شبیه‌سازی را بدون پرش زمانی یا برهم خوردن موقعیت اشیا ادامه می‌دهد.
پایان بازی / شکست `GAME_OVER`	با سقوط توپ از کف صفحه و اتمام ۳ جان رخ می‌دهد. موتور متوقف شده، پیام پایان بازی به نمایش درمی‌آید، بالاترین امتیاز در منبع ذخیره ثبت شده و فشردن کلید آر بازی را از نو شروع می‌کند.
پیروزی بازیکن `VICTORY`	با پاکسازی کامل تمامی ۴۰ آجر فعال می‌شود. پیام تبریک و پیروزی به همراه امتیاز نهایی نمایش داده شده و کلید آر امکان شروع یک دور تازه را فراهم می‌سازد.

گرافیک و واسط کاربری

نمایش بصری بازی بریک‌اوت بر پایه یک بوم دوبعدی شفاف و پایدار استوار است. رعایت چند اصل ساده گرافیک کامپیوتری باعث می‌شود بازی شما کاملاً روان و بدون پرش یا لرزش تصویر اجرا شود:

۱. دستگاه مختصات و هندسه بوم

در محیط‌های گرافیکی رایانه، جهت محورها با دستگاه دکارتی ریاضی کمی تفاوت دارد:

- نقطه مبدا با مختصات $(0, 0)$ در گوشه بالا-چپ بوم قرار گرفته است.
- محور افقی مثبت به سمت راست پنجره پیش می‌رود.
- محور عمودی مثبت به سمت پایین و کف پنجره امتداد دارد.

بنابراین هر زمان توپ رو به بالا و به سمت آجرها حرکت می‌کند، سرعت عمودی آن منفی است؛ و وقتی به سمت سکو در پایین سقوط می‌کند، سرعت عمودی مثبت است.

۲. پایداری تصویر و بافر دوگانه (Double Buffering)

شاید برایتان پیش آمده باشد که در برنامه‌های گرافیکی، تصویر چشمک بزند یا بلرزد. این پدیده به خاطر پاک‌شدن و کشیده‌شدن پیاپی اشیا روی صفحه است:

- با استفاده از بافر دوگانه، تمام نقاشی‌ها ابتدا در یک حافظه پس‌زمینه کامل می‌شوند و سپس فریم کامل‌شده یک‌باره به نمایش درمی‌آید.
- کتابخانه گرافیکی ACM این سازوکار را در بطن خود مدیریت می‌کند تا کار شما برای ایجاد انیمیشن روان و بی‌دردسر باشد.

۳. اجزای دیداری و نوار وضعیت

بخش بصری	مشخصات ترسیم و استانداردهای نمایش
بوم اصلی بازی	مستطیلی به ابعاد ۸۰۰ در ۶۰۰ پیکسل با پس‌زمینه تیره یا یکدست که محدوده شبیه‌سازی را مشخص می‌سازد.
سکو و توپ	سکو به صورت مستطیلی توپر با رنگ متمایز و توپ به شکل دایره‌ای یکدست با قطر ۱۶ پیکسل.
شبکه آجرها	مستطیل‌های رنگی هم‌اندازه با فاصله مرزی ۶ پیکسل؛ تنوع رنگ سطرها بازی را چشم‌نواز می‌کند.
نوار وضعیت (HUD)	نوار بالای صفحه برای نمایش امتیاز، ۳ قلب جان و رکورد، به همراه پیام‌های توقف و پایان در وسط صفحه.

فیزیک و برخوردها

موتور فیزیک بازی وظیفه شبیه‌سازی حرکت پیوسته، محاسبه زاویه بازتاب از سکو و انهدام دقیق آجرها را بر عهده دارد:

۱. بازتاب زاویه‌دار از سکو

اگر توپ همیشه مثل آینه بازتاب کند، بازی سریعاً وارد یک چرخه تکراری و خسته‌کننده می‌شود. برای هیجان بیشتر، زاویه پرتاب با توجه به فاصله نقطه برخورد تا مرکز سکو تغییر می‌کند:

- **محاسبه انحراف نسبی برخورد:**
$d = \frac{x_{\text{ball}} - x_{\text{paddle_center}}}{\frac{W_{\text{paddle}}}{2}} \in [-1.0, +1.0]$
- **زاویه بیشینه پرتاب:**
$\theta = d \times 60^{\circ}$
- **پایستگی تندی کل توپ:**
$v = \sqrt{v_x^2 + v_y^2}$
- **سرعت‌های تفکیک‌شده جدید:**
$v_{y'} = -v\cos(\theta)$ و $v_{x'} = v\sin(\theta)$

علامت مؤلفه عمودی همیشه منفی است تا توپ همیشه رو به بالا و به سمت آجرها شلیک شود.

```
// Calculate hit offset relative to paddle
center (-1.0 to +1.0)
double paddleCenter = paddle.getX() +
paddle.getWidth() / 2.0;
double ballCenter = ball.getX() +
ball.getRadius();
double offset = (ballCenter - paddleCenter) /
(paddle.getWidth() / 2.0);
offset = Math.max(-1.0, Math.min(1.0, offset));

double maxAngle = Math.toRadians(60.0);
double angle = offset * maxAngle;
double speed = Math.hypot(ball.getVx(),
ball.getVy());

ball.setVx(speed * Math.sin(angle));
ball.setVy(-Math.abs(speed * Math.cos(angle)));
```

۲. تشخیص برخورد با آجرها (AABB)

برای بررسی برخورد توپ با آجرها و دیواره‌ها، از الگوریتم تداخل مستطیل‌های محیطی (AABB) استفاده می‌کنیم. به محض برخورد، سرعت عمودی یا افقی معکوس شده، آجر مربوطه ناپدید می‌شود و ۱۰۰ امتیاز شیرین به بازیکن اضافه می‌گردد!

نکات طلایی برای پیاده‌سازی روان فیزیک

۱. **گیر افتادن توپ در سکو:** اگر فقط علامت سرعت را منفی کنید، توپ ممکن است در فریم بعدی همچنان داخل سکو بماند و شروع به لرزیدن کند! چاره کار: موقعیت عمودی توپ را پس از برخورد، مماس با سطح بالایی سکو قرار دهید.

۲. **تقسیم صفر ناشی از اعداد صحیح:** در محاسبه نسبت فاصله، متغیرها باید اعشاری (`double`) باشند؛ در غیر این صورت تقسیم عدد صحیح حاصل را صفر می‌کند.

۳. **پرش شبح‌وار از موانع (Tunneling):** اگر سرعت توپ خیلی زیاد شود، ممکن است در یک فریم از عرض آجر رد شود بدون اینکه برخوردی ثبت شود! سرعت را در اندازه‌ای منطقی نگه دارید.

گام‌های پیاده‌سازی

برای اینکه قدم‌به‌قدم و با آرامش پیش بروید، پیشنهاد می‌کنیم ساخت بازی را به ۵ مرحله مستقل و لذت‌بخش تقسیم کنید:

۱. **گام اول: پنجره، بوم و سکوی بازی**
راه‌اندازی پنجره به ابعاد ۸۰۰ در ۶۰۰ پیکسل، ساخت کلاس سکو و کنترل حرکت افقی با کلیدهای چپ و راست.
معیار پذیرش: سکو به نرمی حرکت کند و با رسیدن به لبه‌های چپ و راست صفحه متوقف شود.

۲. **گام دوم: کلاس توپ، حرکت و بازتاب از دیواره‌ها**
ساخت کلاس توپ، بردار سرعت و بازتاب الاستیک از سه دیواره بالا، چپ و راست بوم.
معیار پذیرش: توپ در ابتدا همراه سکو حرکت کند و با زدن کلید فاصله، شلیک شود.

۳. **گام سوم: بازتاب زاویه‌دار از سکو و سیستم ۳ جان**
فرمول بازتاب زاویه‌دار (بین منفی ۶۰ تا مثبت ۶۰ درجه) و کسر جان با عبور توپ از کف صفحه.
معیار پذیرش: با افتادن توپ، یک قلب کم شود و توپ دوباره روی سکو آماده پرتاب باشد.

۴. **گام چهارم: ماتریس ۴۰ آجر، تشخیص برخورد و امتیاز**
چیدمان آجرها در ۵ سطر و ۸ ستون، الگوریتم برخورد و اضافه شدن ۱۰۰ امتیاز به ازای هر آجر.
معیار پذیرش: اصابت به هر آجر بلافاصله آن را محو کرده و جهت توپ را تغییر دهد.

۵. **گام پنجم: ماشین وضعیت، توقف، برد، باخت و شروع دوباره**
اتصال وضعیت‌های بازی، پیام‌های توقف با کلید P و راه‌اندازی دوباره در هر لحظه با کلید R.
معیار پذیرش: بازی با از دست رفتن جان‌ها یا تخریب تمام آجرها به پایان برسد و کلید R همه‌چیز را ریست کند.

راهبرد توسعه و کامیت‌ها

پیشنهاد می‌کنیم بعد از تکمیل هر گام، بلافاصله یک کامیت استاندارد در گیت‌هاب ثبت کنید. این کار به شما اطمینان می‌دهد که همیشه یک نسخه پایدار و کارآمد در جیب دارید!

معیارهای ارزیابی و دفاع

فاز اول پروژه دارای ۱۰۰ نمره تخصصی است که معادل ۲ نمره از ۲۰ نمره کل پایانی درس محاسبه می‌شود:

معیار ارزیابی	مؤلفه‌ها و جزئیات بررسی فنی	بارم
درستی عملکرد و حس بازی	حرکت روان ۶۰ هرتز، پرتاب دستی با کلید فاصله، بازتاب زاویه‌دار از سکو، بازتاب الاستیک از سه دیواره، تخریب دقیق آجرها با الگوریتم تداخل مستطیل‌ها، چرخه ۳ جان و ماشین وضعیت جامع.	۴۰ نمره
معماری و طراحی شی‌گرا	تفکیک دقیق سه لایه مدل، نما و کنترل‌گر، کپسوله‌سازی کامل فیلدها با سطح دسترسی خصوصی، پایبندی به اصل تک‌مسئولیتی، و بکارگیری رابط قرارداد ذخیره‌سازی داده.	۳۵ نمره
کیفیت کد و مهندسی نرم‌افزار	پرهیز کامل از اعداد جادویی و تجمیع ثابت‌ها در یک کلاس مرجع، نام‌گذاری گویای متغیرها و توابع، ساختار استاندارد بسته‌بندی و تعهد به کامیت‌های منظم در گیت‌هاب.	۱۵ نمره
تسلط و گفت‌وگو در جلسه دفاع	تشریح روان محاسبات هندسی زاویه کمانه، تبیین منطق تشخیص برخورد، توجیه تصمیم‌های شی‌گرایی و پاسخ به پرسش‌های مفهومی دستیاران آموزشی.	۱۰ نمره
مجموع کل ارزیابی	**ارزیابی جامع فاز اول هسته بازی بریک‌اوت**	**۱۰۰ نمره**

جلسه دفاع و گفت‌وگوی فنی

جلسه تحویل پروژه، یک مصاحبه خشک نیست؛ یک گفت‌وگوی صمیمی و دوستانه بین شما و دستیاران آموزشی است تا دستاوردها و تلاش‌هایتان را نشان دهید و بازخوردهای سازنده بگیرید. اگر کدها را خودتان نوشته باشید و چرایی تصمیم‌هایتان را بدانید، این جلسه بسیار لذت‌بخش خواهد بود:

- **تسلط کامل بر کدها:** اگر ساختار کدهای ارسالی را خوب بشناسید و منطق محاسبات فیزیک و کلاس‌ها را روان توضیح دهید، کل نمره را به سادگی دریافت خواهید کرد.
- **اصالت کار و استقلال:** هدف ما اطمینان از یادگیری واقعی شماست. اگر شخصی نتواند دلیل تصمیم‌های کدش را توضیح دهد یا تغییرات کوچکی را در جلسه اعمال کند، نمره تعدیل می‌شود. هوش مصنوعی دستیار خوبی برای یادگیری است، اما مهندس اصلی پروژه خود شمایید.

بخش‌های امتیازی و اختیاری

اگر دوست دارید مهارت‌های خود را فراتر از حد انتظار به چالش بکشید، این دو بخش امتیازی جذاب را امتحان کنید:

- **آجرهای مقاوم دوضربه‌ای (۱۰ نمره امتیازی):** آجرهایی در سطرهای بالا که با ضربه اول ترک برمی‌دارند و با ضربه دوم منهدم می‌شوند (۲۰۰ امتیاز).
- **تعویض پویای راهبرد ذخیره‌ساز (۵ نمره امتیازی):** تغییر بلادرنگ نوع ذخیره رکورد میان حافظه موقت و فایل متنی در حین اجرای بازی.